Logical Operators in MySQL

What are Logical Operators?

Logical Operators are used to combine multiple conditions in a SQL query. They help filter data based on one or more conditions and are commonly used with the WHERE clause.

Types of Logical Operators

- **AND – Returns records only if all conditions are TRUE.**
- **OR – Returns records if at least one condition is TRUE.**
- **NOT – Reverses the result of a condition (TRUE becomes FALSE and vice versa).**

STEP 1: Create Table

STEP 1: Create Table

```
CREATE TABLE Employees (
 id INT,
 name VARCHAR(100),
 department VARCHAR(100),
 salary INT,
 isactive BOOLEAN
 );
```

STEP 2: Insert Records

```
INSERT INTO Employees VALUES
 (1, 'Rahul Sharma', 'IT', 50000, TRUE),
 (2, 'Priya Reddy', 'HR', 30000, TRUE),
 (3, 'Amit Kumar', 'Finance', 45000, FALSE),
 (4, 'Sneha Patel', 'IT', 35000, TRUE),
 (5, 'Vikram Singh', 'Finance', 28000, TRUE),
 (6, 'Anjali Mehta', 'HR', 32000, FALSE),
```

```
(7, 'Kiran Rao', 'IT', 60000, TRUE),
(8, 'Pooja Verma', 'Marketing', 27000, TRUE),
(9, 'Ramesh Naidu', 'Sales', 38000, TRUE),
(10, 'Deepika Joshi', 'Finance', 52000, TRUE),
(11, 'Suresh Babu', 'Marketing', 29000, FALSE),
(12, 'Neha Kapoor', 'HR', 41000, TRUE),
(13, 'Arjun Reddy', 'IT', 55000, FALSE),
(14, 'Lakshmi Devi', 'Sales', 33000, TRUE),
(15, 'Nikhil Jain', 'IT', 47000, TRUE);
```

SELECT Queries

Q1. Display IT department employees whose salary is greater than 40000.

```
SELECT *
 FROM Employees
 WHERE department = 'IT' AND salary > 40000;
```

Q2. Display employees who belong to the HR department or whose salary is less than 35000.

```
SELECT *
 FROM Employees
 WHERE department = 'HR' OR salary < 35000;
```

Q3. Display employees who are not in the HR department.

```
SELECT *
 FROM Employees
 WHERE department != 'HR';
```

Q4. Display employees whose department is Finance or whose salary is greater than 30000 and are active.

```
SELECT *
 FROM Employees
```

```
WHERE department = 'Finance'
OR (salary > 30000 AND isactive = TRUE);
```

Q5. Display active IT employees.

```
SELECT *
FROM Employees
WHERE department = 'IT' AND isactive = TRUE;
```

UPDATE Queries

Q1. Increase salary by 2000 for active IT employees earning less than 60000.

```
UPDATE Employees
SET salary = salary + 2000
WHERE department = 'IT'
AND isactive = TRUE
AND salary < 60000;
```

Q2. Reduce salary by 10% for Finance department employees or inactive employees.

```
UPDATE Employees
SET salary = salary - (salary * 0.10)
WHERE department = 'Finance'
OR isactive = FALSE;
```

Q3. Make all non-IT employees inactive.

```
UPDATE Employees
SET isactive = FALSE
WHERE department != 'IT';
```

Q4. Increase salary by 3000 for active HR employees.

```
UPDATE Employees
 SET salary = salary + 3000
 WHERE department = 'HR'
 AND isactive = TRUE;
```

Q5. Increase salary by 5000 for Sales employees earning less than 40000.

```
UPDATE Employees
 SET salary = salary + 5000
 WHERE department = 'Sales'
 AND salary < 40000;
```

DELETE Queries

Q1. Delete inactive employees from the HR department.

```
DELETE FROM Employees
 WHERE department = 'HR'
 AND isactive = FALSE;
```

Q2. Delete employees from Finance or employees earning less than 35000.

```
DELETE FROM Employees
 WHERE department = 'Finance'
 OR salary < 35000;
```

Q3. Delete employees who are not in the IT department.

```
DELETE FROM Employees
 WHERE department != 'IT';
```

Q4. Delete inactive employees.

```
DELETE FROM Employees
 WHERE isactive = FALSE;
```

Q5. Delete Marketing employees with salary less than 30000.

```
DELETE FROM Employees
 WHERE department = 'Marketing'
 AND salary < 30000;
```

NULL Operator

What are NULL Operators?

NULL Operators are used to check whether a column contains a **NULL** value (missing or unknown data).

> **Note:** `NULL` is **not** the same as `0` (zero) or an empty string (`''`). It represents the absence of a value.

MySQL provides two NULL operators:

- **IS NULL** – Checks whether a column contains a NULL value.
- **IS NOT NULL** – Checks whether a column contains a non-NULL value.

STEP 1: Create Table

```
CREATE TABLE Projects (
 id INT,
 name VARCHAR(100),
 assignedto VARCHAR(100),
 start DATE,
```

```
end DATE
);
```

STEP 2: Insert Records

```
INSERT INTO Projects VALUES
 (1, 'Website Revamp', 'Alice', '2024-01-15', '2024-03-01'),
 (2, 'Mobile App', 'Bob', '2024-02-01', NULL),
 (3, 'Data Migration', NULL, NULL, NULL),
 (4, 'SEO Optimization', 'Eve', '2024-03-10', '2024-04-20'),
 (5, 'Cloud Migration', NULL, '2024-03-15', NULL),
 (6, 'ERP Development', 'John', '2024-04-01', NULL),
 (7, 'CRM Upgrade', NULL, NULL, NULL),
 (8, 'Inventory System', 'David', '2024-05-10', '2024-06-25'),
 (9, 'Online Portal', 'Sophia', '2024-06-15', NULL),
 (10, 'Security Audit', NULL, '2024-07-01', NULL),
 (11, 'AI Chatbot', 'James', NULL, NULL),
 (12, 'Payroll System', 'Emma', '2024-08-01', '2024-09-10'),
 (13, 'Hospital Management', NULL, NULL, NULL),
 (14, 'Student Portal', 'Michael', '2024-09-05', NULL),
 (15, 'Banking Application', NULL, '2024-10-01', NULL);
```

SELECT Queries

Q1. Display all projects that are not assigned to anyone.

```
SELECT *
 FROM Projects
 WHERE assignedto IS NULL;
```

Q2. Display projects that have not started yet.

```
SELECT *
 FROM Projects
 WHERE start IS NULL;
```

Q3. Display all completed projects.

```
SELECT *
 FROM Projects
 WHERE end IS NOT NULL;
```

Q4. Display all projects with an assigned employee.

```
SELECT *
 FROM Projects
 WHERE assignedto IS NOT NULL;
```

Q5. Display projects that have not yet been completed.

```
SELECT *
 FROM Projects
 WHERE end IS NULL;
```

UPDATE Queries

Q1. Assign all unassigned projects to Frank.

```
UPDATE Projects
 SET assignedto = 'Frank'
 WHERE assignedto IS NULL;
```

Q2. Update today's date as the start date for projects that have not started.

```
UPDATE Projects
 SET start = CURDATE()
 WHERE start IS NULL;
```

Q3. Rename all completed projects as 'Completed'.

```
UPDATE Projects
 SET name = 'Completed'
 WHERE end IS NOT NULL;
```

Q4. Set today's date as the end date for projects that have no end date.

```
UPDATE Projects
 SET end = CURDATE()
 WHERE end IS NULL;
```

Q5. Assign 'Admin' to projects without an assigned employee.

```
UPDATE Projects
 SET assignedto = 'Admin'
 WHERE assignedto IS NULL;
```

DELETE Queries

Q1. Delete all unassigned projects.

```
DELETE FROM Projects
 WHERE assignedto IS NULL;
```

Q2. Delete projects that have not started.

```
DELETE FROM Projects
 WHERE start IS NULL;
```

Q3. Delete all completed projects.

```
DELETE FROM Projects
 WHERE end IS NOT NULL;
```

Q4. Delete projects that do not have an end date.

```
DELETE FROM Projects
 WHERE end IS NULL;
```

Q5. Delete projects where both assigned employee and start date are NULL.

```
DELETE FROM Projects
 WHERE assignedto IS NULL
 AND start IS NULL;
```

Other Operators

MySQL offers various additional operators for specific purposes.

BETWEEN: Checks if a value falls within a specific range.

IN: Checks if a value is present within a list of specified values.

CASE: Used for conditional expressions and assigning values based on conditions
BETWEEN Operator

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-03-15' AND '2024-03-20';
```

±This retrieves all orders placed between March 15th, 2024 (inclusive) and March 20th, 2024 (inclusive).

BETWEEN:

Checks if a value falls within a specific range.

STEP 1: Create Table

```
CREATE TABLE Orders (
 id INT,
 customer_name VARCHAR(100),
 order_date DATE,
 total_amount DECIMAL(10,2)
 );
```

STEP 2: Insert Records

```
INSERT INTO Orders VALUES
 (1, 'Alice', '2024-03-01', 1500.00),
 (2, 'Bob', '2024-03-05', 450.00),
 (3, 'Charlie', '2024-03-10', 2500.00),
 (4, 'Diana', '2024-03-15', 800.00),
 (5, 'Eve', '2024-03-20', 1200.00),
 (6, 'Frank', '2024-03-25', 3000.00),
 (7, 'Grace', '2024-04-02', 1800.00),
 (8, 'Henry', '2024-04-08', 950.00),
 (9, 'Ivy', '2024-04-12', 2750.00),
 (10, 'Jack', '2024-04-18', 650.00),
 (11, 'Kevin', '2024-04-22', 4200.00),
 (12, 'Lily', '2024-05-01', 1350.00),
 (13, 'Michael', '2024-05-08', 5000.00),
 (14, 'Nancy', '2024-05-15', 2250.00),
 (15, 'Oliver', '2024-05-25', 900.00);
```

SELECT Queries

Q1. Display orders placed between '2024-03-03' and '2024-03-10'.

```
SELECT *
 FROM Orders
 WHERE order_date BETWEEN '2024-03-03' AND '2024-03-10';
```

Q2. Display orders placed between '2024-04-01' and '2024-04-30'.

```
SELECT *
 FROM Orders
 WHERE order_date BETWEEN '2024-04-01' AND '2024-04-30';
```

Q3. Display orders whose total amount is between 1000 and 3000.

```
SELECT *
 FROM Orders
 WHERE total_amount BETWEEN 1000 AND 3000;
```

Q4. Display orders whose total amount is between 500 and 1500.

```
SELECT *
 FROM Orders
 WHERE total_amount BETWEEN 500 AND 1500;
```

Q5. Display orders placed between '2024-05-01' and '2024-05-31'.

```
SELECT *
 FROM Orders
 WHERE order_date BETWEEN '2024-05-01' AND '2024-05-31';
```

UPDATE Queries

Q1. Increase the total amount by 1000 for orders placed between '2024-03-04' and '2024-03-10'.

```
UPDATE Orders
 SET total_amount = total_amount + 1000
 WHERE order_date BETWEEN '2024-03-04' AND '2024-03-10';
```

Q2. Increase the total amount by 500 for orders placed between '2024-04-01' and '2024-04-15'.

```
UPDATE Orders
 SET total_amount = total_amount + 500
 WHERE order_date BETWEEN '2024-04-01' AND '2024-04-15';
```

Q3. Reduce the total amount by 200 for orders between 1000 and 2000.

```
UPDATE Orders
 SET total_amount = total_amount - 200
 WHERE total_amount BETWEEN 1000 AND 2000;
```

Q4. Increase the total amount by 10% for orders between 2500 and 5000.

```
UPDATE Orders
 SET total_amount = total_amount + (total_amount * 0.10)
 WHERE total_amount BETWEEN 2500 AND 5000;
```

Q5. Add 300 to orders placed between '2024-05-01' and '2024-05-31'.

```
UPDATE Orders
 SET total_amount = total_amount + 300
 WHERE order_date BETWEEN '2024-05-01' AND '2024-05-31';
```

DELETE Queries

Q1. Delete orders placed between '2024-03-10' and '2024-03-20'.

```
DELETE FROM Orders
 WHERE order_date BETWEEN '2024-03-10' AND '2024-03-20';
```

Q2. Delete orders placed between '2024-04-01' and '2024-04-15'.

```
DELETE FROM Orders
 WHERE order_date BETWEEN '2024-04-01' AND '2024-04-15';
```

Q3. Delete orders whose total amount is between 500 and 1000.

```
DELETE FROM Orders
 WHERE total_amount BETWEEN 500 AND 1000;
```

Q4. Delete orders whose total amount is between 2500 and 5000.

```
DELETE FROM Orders
 WHERE total_amount BETWEEN 2500 AND 5000;
```

Q5. Delete orders placed between '2024-05-01' and '2024-05-31'.

```
DELETE FROM Orders
 WHERE order_date BETWEEN '2024-05-01' AND '2024-05-31';
```

IN Operator

This operator checks if a value exists within a specified list of values

```
SELECT *
FROM customers
WHERE country IN ('USA', 'Canada', 'UK');
```

±This retrieves customer records where the country is either 'USA', 'Canada', or 'UK'.

STEP 1: Create Table

```
CREATE TABLE Emp (
 id INT,
```

```
name VARCHAR(100),
dept VARCHAR(20),
salary INT
);
```

STEP 2: Insert Records

```
INSERT INTO Emp VALUES
(1, 'Rahul', 'IT', 50000),
(2, 'Ramesh', 'HR', 35000),
(3, 'Karan', 'Finance', 45000),
(4, 'Mahesh', 'Marketing', 40000),
(5, 'Suresh', 'IT', 55000),
(6, 'Arjun', 'Sales', 38000),
(7, 'Manish', 'Admin', 42000),
(8, 'Ganesh', 'Finance', 47000),
(9, 'Naresh', 'HR', 39000),
(10, 'Rohan', 'IT', 52000),
(11, 'Mohan', 'Sales', 41000),
(12, 'Kishan', 'Marketing', 43000),
(13, 'Dinesh', 'Admin', 46000),
(14, 'Rajesh', 'Finance', 51000),
(15, 'Kiran', 'IT', 48000);
```

SELECT Queries

Q1. Display employees from the IT and Finance departments.

```
SELECT *
FROM Emp
WHERE dept IN ('IT', 'Finance');
```

Q2. Display employees from the HR and Sales departments.

```
SELECT *
 FROM Emp
 WHERE dept IN ('HR', 'Sales');
```

Q3. Display employees from the Marketing and Admin departments.

```
SELECT *
 FROM Emp
 WHERE dept IN ('Marketing', 'Admin');
```

Q4. Display employees from the HR, IT, and Finance departments.

```
SELECT *
 FROM Emp
 WHERE dept IN ('HR', 'IT', 'Finance');
```

Q5. Display employees whose salary is 35000, 42000, or 60000.

```
SELECT *
 FROM Emp
 WHERE salary IN (35000, 42000, 60000);
```

UPDATE Queries

Q1. Increase salary by 2000 for employees in the HR or Marketing departments.

```
UPDATE Emp
 SET salary = salary + 2000
 WHERE dept IN ('HR', 'Marketing');
```

Q2. Increase salary by 3000 for employees in the IT and Finance departments.

UPDATE Emp
SET salary = salary + 3000
WHERE dept IN ('IT', 'Finance');

Q3. Increase salary by 1500 for employees in the Sales and Admin departments.

UPDATE Emp
SET salary = salary + 1500
WHERE dept IN ('Sales', 'Admin');

Q4. Increase salary by 5000 for employees earning 35000, 39000, or 43000.

UPDATE Emp
SET salary = salary + 5000
WHERE salary IN (35000, 39000, 43000);

Q5. Increase salary by 1000 for employees in the HR, IT, and Sales departments.

UPDATE Emp
SET salary = salary + 1000
WHERE dept IN ('HR', 'IT', 'Sales');

DELETE Queries

Q1. Delete employees from the Finance or Marketing departments.

DELETE FROM Emp
WHERE dept IN ('Finance', 'Marketing');

Q2. Delete employees from the HR department.

DELETE FROM Emp
WHERE dept IN ('HR');

Q3. Delete employees from the Sales and Admin departments.

DELETE FROM Emp
WHERE dept IN ('Sales', 'Admin');

Q4. Delete employees whose salary is 35000, 39000, or 43000.

DELETE FROM Emp
WHERE salary IN (35000, 39000, 43000);

Q5. Delete employees from the IT department.

DELETE FROM Emp
WHERE dept IN ('IT');

LIKE Operator Practice Questions

SELECT Queries (LIKE Operator)

Q1. Display employees whose names start with 'R'.

SELECT *
FROM Emp
WHERE name LIKE 'R%';

Q2. Display employees whose names end with 'sh'.

```
SELECT *
 FROM Emp
 WHERE name LIKE '%sh';
```

Q3. Display employees whose names contain 'ar'.

```
SELECT *
 FROM Emp
 WHERE name LIKE '%ar%';
```

Q4. Display employees whose names have exactly 5 characters.

```
SELECT *
 FROM Emp
 WHERE name LIKE '_____';
```

– Q5. Display employees whose names start with 'K' and end with 'n'.

SELECT

```
SELECT *
 FROM Emp
 WHERE name LIKE 'K%n';
```

UPDATE Queries

Q1. Increase salary by 2000 for employees whose names start with 'R'.

```
UPDATE Emp
 SET salary = salary + 2000
 WHERE name LIKE 'R%';
```

Q2. Increase salary by 3000 for employees whose names end with 'sh'.

```
UPDATE Emp
 SET salary = salary + 3000
 WHERE name LIKE '%sh';
```

Q3. Increase salary by 1000 for employees whose names contain 'ar'.

```
UPDATE Emp
 SET salary = salary + 1000
 WHERE name LIKE '%ar%';
```

Q4. Increase salary by 500 for employees whose names have exactly 5 characters.

```
UPDATE Emp
 SET salary = salary + 500
 WHERE name LIKE '_____';
```

Q5. Increase salary by 2500 for employees whose names start with 'K' and end with 'n'.

```
UPDATE Emp
 SET salary = salary + 2500
 WHERE name LIKE 'K%n';
```

DELETE Queries

Q1. Delete employees whose names start with 'R'.

```
DELETE FROM Emp
 WHERE name LIKE 'R%';
```

Q2. Delete employees whose names end with 'sh'.

DELETE FROM Emp
WHERE name LIKE '%sh';

Q3. Delete employees whose names contain 'ar'.

DELETE FROM Emp
WHERE name LIKE '%ar%';

Q4. Delete employees whose names have exactly 5 characters.

DELETE FROM Emp
WHERE name LIKE '_____';

Q5. Delete employees whose names start with 'K' and end with 'n'.

DELETE FROM Emp
WHERE name LIKE 'K%n';

CASE Statement in MySQL

The CASE statement is used to apply conditional logic in SQL. It works similarly to the if-else statement in programming languages.

Syntax

```
CASE
   WHEN condition1 THEN result1
   WHEN condition2 THEN result2
   ...
   ELSE result
END
```

Sample Table

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    department VARCHAR(30),
    salary DECIMAL(10,2),
    experience INT,
    age INT
);

INSERT INTO employees VALUES
(101,'Ravi','IT',65000,5,28),
(102,'Priya','HR',45000,2,24),
(103,'Amit','Finance',75000,8,35),
(104,'Sneha','IT',55000,3,27),
(105,'Rahul','Sales',40000,1,22),
(106,'Kiran','Finance',90000,12,40),
(107,'Anjali','HR',52000,4,29),
(108,'Suresh','Sales',68000,7,33);
```

CASE Examples

Example 1: Employee Salary Category

```
SELECT emp_name,
salary,
CASE
    WHEN salary >= 80000 THEN 'High Salary'
    WHEN salary >= 60000 THEN 'Medium Salary'
    ELSE 'Low Salary'
END AS salary_category
FROM employees;
```

Output

Employee	Salary	Category
Ravi	65000	Medium Salary

Priya	45000	Low Salary
Amit	75000	Medium Salary
Sneha	55000	Low Salary
Rahul	40000	Low Salary
Kiran	90000	High Salary
Anjali	52000	Low Salary
Suresh	68000	Medium Salary

Example 2: Experience Level

```
SELECT emp_name,
experience,
CASE
   WHEN experience >= 10 THEN 'Expert'
   WHEN experience >= 5 THEN 'Senior'
   WHEN experience >= 2 THEN 'Junior'
   ELSE 'Fresher'
END AS experience_level
FROM employees;
```

Example 3: Department BonusSELECT emp_name,

```
department,
salary,
CASE
   WHEN department='IT' THEN salary*0.20
   WHEN department='Finance' THEN salary*0.15
   WHEN department='HR' THEN salary*0.10
   ELSE salary*0.05
END AS bonus
FROM employees;
```

Example 4: Voting Eligibility

```
SELECT emp_name,
age,
CASE
   WHEN age>=18 THEN 'Eligible'
   ELSE 'Not Eligible'
END AS voting_status
FROM employees;
```

Example 5: Grade Employees

```
SELECT emp_name,
salary,
CASE
   WHEN salary>=80000 THEN 'A'
   WHEN salary>=60000 THEN 'B'
   WHEN salary>=50000 THEN 'C'
   ELSE 'D'
END AS grade
FROM employees;
```

Example 6: CASE in ORDER BY

Display IT employees first.

```
SELECT *

FROM employees
ORDER BY
CASE
   WHEN department='IT' THEN 1
   ELSE 2
END;
```

Example 7: CASE in UPDATE

Increase salary based on experience.

```
UPDATE employees
SET salary =
CASE
   WHEN experience>=10 THEN salary+10000
   WHEN experience>=5 THEN salary+5000
   ELSE salary+2000
END;
```

Example 8: CASE with Aggregate Function

```
SELECT
SUM(
CASE
   WHEN department='IT'
   THEN salary
   ELSE 0
END
) AS total_it_salary
FROM employees;
```

Interview Questions (CASE)

Q1. Display employee name and salary status.

Q2. Display "Experienced" if experience is more than 5 years.